Week 03 — Chapter II: Process Management

> Slides: week03-chapter-2-process-management.pdf

II.1. Overview

In operating systems, **processes and threads are the fundamental units of execution**. This chapter covers their **structure, behavior, and management**, to understand how **multitasking and concurrency** work.

Outline

1. What is a Process?
2. What is a Thread?
3. Processes vs Threads
4. Implementation of Processes and Threads
5. Conclusion

II.2. What is a Process?

> A **process** is an **instance of a program in execution**.

A program is a file on disk. A **process** is that program **running**, with state attached to it. It consists of:

- **Program code** (text section)
- **Current activities** (program counter, registers)
- **Associated resources** (memory, files, CPU state)

Characteristics:

Property	Meaning
Isolation	Each process has its **own memory space**
Heavyweight	Requires **more system resources**
Context switching	**Slower**, because of memory isolation
Communication	Needs an **inter-process communication (IPC)** mechanism

What processes have

Attribute	Meaning
Process ID (PID)	Unique numerical identifier
Program counter	The next instruction to execute
State	New, ready, running, waiting, terminated
Priority	Determines CPU access priority
I/O status	List of I/O devices allocated

The five **states** are worth memorising as a life cycle: a process is **new** when created, **ready** when it could run but the CPU is busy, **running** when it is on the CPU, **waiting** when it needs something (usually I/O) before it can continue, and **terminated** when it has finished.

II.2.1. Types of process

Type	Description
Foreground	Interacts directly with the user
Background	Runs without user interaction
Daemon	Background service that starts at boot
Parent	Process that creates other processes
Child	Process created by another process
Zombie	A **terminated** process that still has an entry in the process table
Orphan	A child process whose **parent has terminated**

A **zombie** has finished but its exit status hasn't been collected by the parent yet, so the table entry stays. An **orphan** is still running but has lost its parent, and the OS re-parents it (on Unix, to `init` / `systemd`).

II.3. What is a Thread?

> A **thread** is the **smallest unit of execution within a process**, sharing the process's memory and resources.

Characteristics:

Property	Meaning
Shared memory	Threads in the same process **share memory**
Lightweight	**Less overhead** than a process
Faster context switching	Because memory is shared
Simplified communication	**No IPC needed**

One process can hold many threads. They share the same address space, so they can exchange data by simply reading the same variables, which is both the advantage (fast) and the danger (they can corrupt each other's data).

II.3.1. Types of threads

Type	Description	Example
User-level	Managed by **user-level libraries**; fast but **not visible to the OS**	Video game engine threads managing rendering. **If one blocks, the game can bug**
Kernel-level	Managed **directly by the OS**; slower but powerful	Multiple client requests handled by a web server. **If one blocks, others can still continue**
Hybrid	Combines user- and kernel-level management	Used by **Solaris** OS

The key contrast: the OS can't see user-level threads, so **one blocking call blocks the whole process**. Kernel-level threads are scheduled individually, so a block affects only that thread.

II.4. Processes vs Threads

Feature	Process	Thread
Memory	Isolated	Shared
Creation overhead	High	Low
Communication	Requires IPC	Direct, via shared memory
Context switching	Slower	Faster
Resource usage	More	Less
Termination	Slower	Faster

Trade-off in one line: **processes are safer (isolation), threads are cheaper (sharing)**.

II.5. Implementation of Processes and Threads

II.5.1. Multiprocessing

> A programming technique where **multiple independent processes** are created, each with its **own dedicated memory space**.

Characteristics:

- **Separate memory:** processes do not share memory by default.
- **Heavyweight:** creating and destroying processes is **slower** and uses **more resources** than threads.
- **True parallelism:** each process has its own interpreter and memory space, so the OS can schedule them to run **simultaneously on multiple CPU cores**.

- **Best for: CPU-bound tasks**, where the goal is to speed up computation by dividing work across cores.

II.5.2. Threading

> A programming technique where **multiple threads** exist **within a single process**, sharing its memory and resources.

Characteristics:

- **Shared memory:** all threads can access and modify the same global variables and data structures.
- **Lightweight:** creating and destroying threads is faster and uses fewer resources.
- **Concurrency, not always parallelism:** threads are often used for **I/O-bound** tasks — while one thread waits, others can run.
- **Best for: I/O-bound tasks**, responsive user interfaces, and tasks needing frequent, fast communication between concurrent units.

How to choose:

If the work is...	Use	Why
CPU-bound (calculation)	**Multiprocessing**	Real parallelism across cores
I/O-bound (waiting on disk, network, user)	**Threading**	Cheap, and waiting threads don't need a core

Concurrency vs parallelism: *concurrency* means several tasks are in progress and take turns; *parallelism* means they literally run at the same instant on different cores.

II.7. Conclusion

The chapter covered **processes and threads**, the building blocks of **multitasking and concurrency**. These concepts explain how an OS manages execution, and they matter for designing **efficient, responsive, and scalable** applications across platforms.

Lab

> **Objective:** create a dashboard to monitor **CPU and memory usage in real time**. **Tools:** `psutil` (system info), `tkinter` (GUI), `matplotlib` (graph).

This is the lab in assignment/ (hello.py, unknown.py).

How it connects to the chapter:

- `psutil.cpu_percent()` and `psutil.virtual_memory()` read the data the OS keeps **about processes** — the same accounting described in II.2.
- The dashboard updates with `root.after(1000, update)` instead of a loop that sleeps, because the tkinter GUI thread must stay **responsive**. That is the **I/O-bound / responsive UI** case from II.5.2, where you keep the waiting off the thread that draws the window.
- A long calculation in the same thread would freeze the window, which is exactly the symptom used in the week 04 practice questions.

Most important for exams

- **Process = an instance of a program in execution**: code (text), current activity (program counter, registers), and resources (memory, files, CPU state).
- **Process attributes:** PID, program counter, **state**, priority, I/O status.
- **Five states:** new → ready → running → waiting → terminated.
- **Process types:** foreground, background, **daemon** (starts at boot), parent, child, **zombie** (terminated but still in the process table), **orphan** (parent terminated).
- **Thread = smallest unit of execution inside a process**, sharing its memory. Lightweight, faster context switching, **no IPC needed**.
- **Thread types:** user-level (library-managed, invisible to the OS, one block can stall everything), kernel-level (OS-managed, others continue), hybrid (Solaris).
- **Processes vs threads table:** memory isolated/shared, overhead high/low, communication IPC/shared memory, switching slower/faster, resources more/less, termination slower/faster.
- **Multiprocessing → CPU-bound, true parallelism. Threading → I/O-bound, concurrency, responsive UIs.**
- **Isolation vs sharing** is the trade-off behind every row of that table.